

Cours Informatique

Programmation Langage C

SMP4

Pr.RAOUYANE Brahim

Partie II

- ▶ Séance 5 : Types composés (1) : Tableaux
- ▶ **Séance 6 : Fonctions**
 - ▶ **Fonctions** : *Fonction, Procédure*
- ▶ **Séance 7 : Pointeurs**
- ▶ **Séance 8 : Types composés (2)**
 - ▶ **Variables**
 - ▶ Structures
 - ▶ Champs de bits
 - ▶ Unions
 - ▶ **Constantes**
 - ▶ Énumérations
 - ▶ **Définition de types: typedef**

Fonctions

Généralités

Structure d'un programme C

```
#include <stdio.h>
```

```
#define constante
```

déclaration de fonctions et de variables

```
main()
```

```
{
```

déclaration de fonctions et de variables
instructions

```
}
```

```
fct1()
```

```
{
```

déclaration de fonctions et de variables
instructions

```
}
```

```
fct2()
```

```
{.....
```

```
}
```

Zone des
déclarations

Zone des
fonctions

Fonctions

- ▶ **Pourquoi?**
- ▶ **C'est quoi la différence entre Fonction et Procédure**
- ▶ **Comment écrire un sous Programme?**
- ▶ **Réversivité**

Fonctions: Définition

► Sous-programme, Sous-Algorithmes: Notions

► Définition:

- Un **sous-algorithme** est un élément d'algorithme nommé et éventuellement paramétré que l'on définit afin de pouvoir ensuite l'appeler par son nom en affectant, s'il y a lieu, des valeurs aux paramètres.

► Intérêt :

- Réaliser un découpage d'une tâche en **sous-tâche**.
- **Effectuer une seule description** d'une tâche commune.
- Concevoir une application de manière descendante en entrant de plus en plus dans les détails.

► Structure : un sous-algorithme est composé

► D'une tête:

- **Nom** sous-algorithme
- **Paramètres(arguments) avec leur type**

- **D'un corps:** des déclarations d'objets locaux aux sous-algorithmes, instructions à exécuter

Fonctions: Définition

- ▶ Les **fonctions et les procédures sont des modules** ont plusieurs avantages :
 - ▶ **Permettent d'éviter de réécrire un même traitement plusieurs fois.**
 - ▶ On fait appelle à **un sous algorithme aux endroits spécifiés.**
 - ▶ Permettent **d'organiser le code et améliorent la lisibilité des programmes**
 - ▶ **Facilitent la maintenance du code (il suffit de modifier une seule fois)**
 - ▶ Ces procédures et fonctions peuvent éventuellement **être réutilisées** dans d'autres programmes

Programmation procédurale

Programmation modulaire

Fonctions: Définition

► **Exemple:**

- *Écrire un programme qui affiche en ordre croissant les notes d'une classe suivies de la note la plus faible, de la note la plus élevée et de la moyenne.*

► **Solution :**

- Revient à résoudre les sous problèmes suivants :
 - 1) Remplir un tableau de naturels avec des notes saisies par l'utilisateur
 - 2) Afficher un tableau de naturels
 - 3) Trier un tableau de naturel en ordre croissant
 - 4) Trouver le plus petit naturel d'un tableau
 - 5) Trouver le plus grand naturel d'un tableau
 - 6) Calculer la moyenne d'un tableau de naturels

► **Intérêts :**

- **Découpage** en Fonctions
- Chaque Fonction peut être **développée** **appart**
- Possibilité de réutilisation de chaque fonction **indépendamment**

Fonctions: Définition

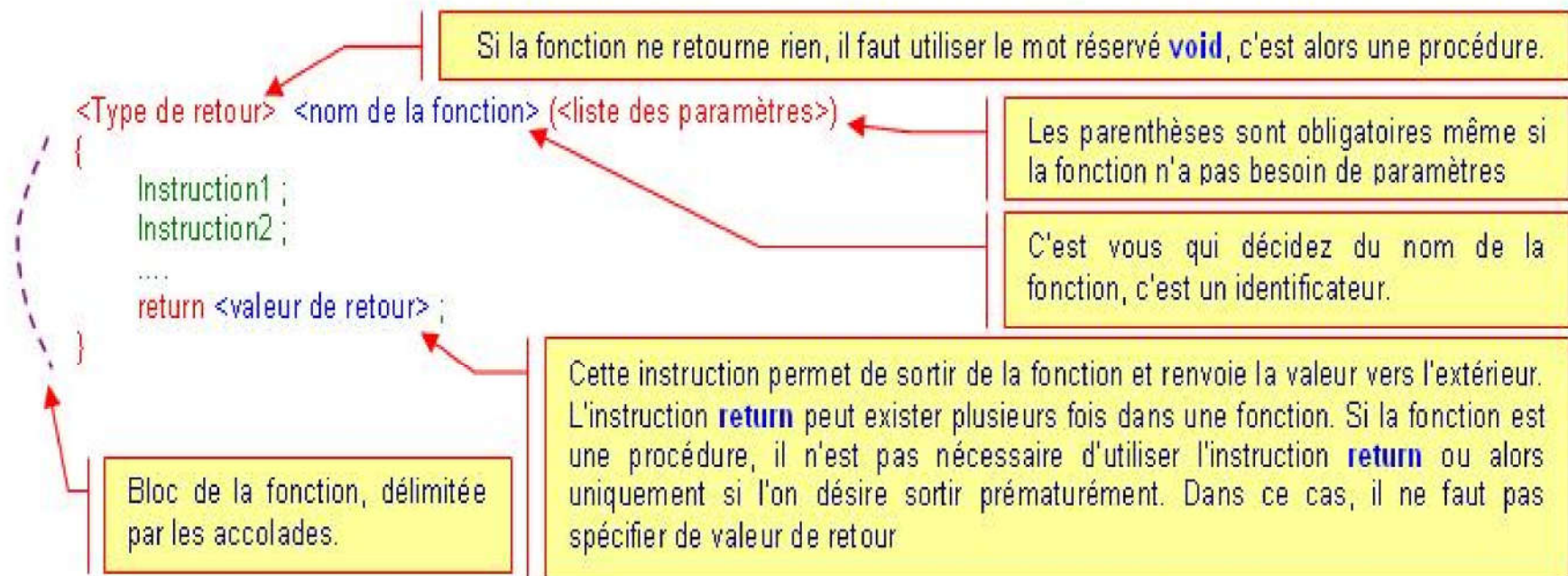
► Définition d'une fonction

- Une **fonction** peut s'apparenter à une **opération** par l'utilisateur.
- Une **fonction** est représentée par un **nom**.
- Les **opérandes** d'une fonction, appelée **paramètres**, sont spécifiés dans une liste entourée de **parenthèses**, les paramètres étant séparés par des **virgules**.
- Le **résultat d'une fonction** se nomme valeur de **retour et le type** de la valeur de retour s'appelle type de retour.
- Une **fonction** ne renvoyant pas de valeur a un type de retour **void**, ce qui veut dire qu'elle ne renvoie rien.
- Dans ce cas particulier, cette fonction s'appelle une **procédure**. **Les actions qu'exécute une fonction sont spécifiées dans le corps de la fonction.**
- Le **type de retour** de la fonction suivi du **nom de la fonction**, la liste des **paramètres** et le **corps** de la fonction, composent la définition de la fonction.

Fonctions: Ecriture

► Comment écrire un sous Programme?

- **Fonction**
- **Procédure**



Fonctions: Ecriture

- ▶ Lors de **l'appel d'une fonction** il y a **suspension** de l'exécution de la fonction en cours.
- ▶ On « **saute** » à l'exécution de la fonction appelée. Quand l'évaluation de la fonction appelée est terminée, la fonction suspendue reprend son exécution à l'endroit qui suit immédiatement l'appel.
- ▶ L'exécution d'une fonction se termine une fois exécutée la dernière instruction du corps de la fonction ou quand une instruction `return` est rencontrée dans le corps de la fonction.

```
void Affiche();  
int Calcul(int valeur);
```

```
void main()  
{  
    int temp,resultat;  
    char caract;  
    float var;  
    ...  
    resultat = Calcul(temp);  
    ...  
    Affiche();  
    ...  
    ...  
}
```

Lors de l'appel de `Calcul`, l'exécution de la fonction principale est interrompue. La fonction `Calcul` est alors exécutée jusqu'à la rencontre de l'instruction `return`. La fonction principale reprend alors son cours et récupère la valeur retournée pour la placer dans la variable `resultat`.

```
int Calcul(int valeur)  
{  
    ...  
    return quelqueChose;  
}  
  
void Affiche()  
{  
    ...  
}
```

Fonctions: Ecriture

<Type de retour> <nom de la fonction> (<liste des paramètres>) // Signature d'une fonction

-3-

-1-

```
int produit (int, int );

main() {
    int a = 2, b = 5;
    printf("%d\n", produit(a,b));
}

int produit (int a, int b) {
    return(a*b); }
```

-2-

```
int produit (int a, int b) {
    return(a*b); }

main() {
    int a = 2, b = 5;
    printf("%d\n", produit(a,b));
}
```

//File PRO.h

```
int produit (int, int );
```

//File PRO.c

```
#include " PRO.h "
```

```
int produit (int a, int b) {
    return(a*b); }
```

// Programme Principale

```
#include " PRO.h "
#include <stdio.h>
```

```
main() {
    int a = 2, b = 5;
    printf("%d\n", produit(a,b));
}
```

Fonctions: Exercice

- ▶ **On dispose d'un tableau T d'entier de taille nb**
 - ▶ Ecrire un **sous-programme** qui permet le remplissage d'un tableau.
 - ▶ Ecrire un **sous-programme** qui permet l'affichage d'un tableau.
 - ▶ Ecrire des **sous-programme** qui retourne la valeur de nombres Max, Min et Moy
- ▶ **Choix entre Fonction et Procédure**
- ▶ **Savoir les signature des fonctions et Procédure**
- ▶ **Savoir les instructions dans chaque F/P**

Fonctions: Exercice

```
void main()
{
float Tab [100] ;
int nb ;
printf( "Combien de valeurs ?");
scanf("%d",&nb) ;
ChargerTab(nb, Tab) ;
EditerTab ( nb,Tab) ;

printf( "Somme = %f \n" ,SommeTab(nb,Tab));
printf ( "Moyenne = %f \n" ,MoyenneTab(nb ,Tab)) ;
printf ( "Max = %f \n" ,MaxTab(nb,Tab ));
printf( "Min = %f \n" ,MinTab (nb,Tab));
}
```


Fonctions: Exercice

```
void main()
{
float Tab [100] ;
int nb ;
printf( "Combien de valeurs ?");
scanf("%d",&nb) ;
ChargerTab(nb, Tab) ;
EditerTab ( nb,Tab) ;
printf( "Somme= %f \n" ,SommeTab(nb,Tab));
printf ( "Moyenne= %f \n" ,MoyenneTab(nb ,Tab)) ;
printf ( "Max= %f \n" ,MaxTab(nb,Tab )) ;
printf( "Min= %f \n" ,MinTab (nb,Tab));
}
```

```
void ChargerTab(int N,float Tab[]){
    int i;
    for(i=0;i<N;i++){
        printf("Valeur %d: ",i+1);
        scanf("%f",&Tab[i]);
    }
    printf("\n");
}
```

```
void EditerTab(int N,float Tab[]){
    int i;
    for(i=0;i<N;i++)
        printf("Valeur %d = %f\n",i+1,Tab[i]);
    printf("\n");
}
```

Fonctions: Exercice

```
void main()
{
float Tab [100] ;
int nb ;
printf( "Combien de valeurs ?");
scanf("%d",&nb) ;
ChargerTab(nb, Tab) ;
EditerTab ( nb,Tab) ;
printf( "Somme= %f \n" ,SommeTab(nb,Tab));
printf ( "Moyenne= %f \n" ,MoyenneTab(nb ,Tab)) ;
printf ( "Max= %f \n" ,MaxTab(nb,Tab )) ;
printf( "Min= %f \n" ,MinTab (nb,Tab));
}
```

```
float SommeTab(int N,float Tab[]){
    int i;
    float resultat=0;
    for(i=0;i<N;i++)
        resultat+=Tab[i];
    return resultat;
}
```

```
float MoyenneTab(int N,float Tab[]){
    return SommeTab(N,Tab)/N;
}
```

Fonctions: Exercice

```
void main()
{
float Tab [100] ;
int nb ;
printf( "Combien de valeurs ?");
scanf("%d",&nb) ;
ChargerTab(nb, Tab) ;
EditerTab ( nb,Tab) ;
printf( "Somme_==_%f \n" ,SommeTab(nb,Tab));
printf ( "Moyenne_==_%f \n" ,MoyenneTab(nb ,Tab)) ;
printf ( "Max_==_%f \n" ,MaxTab(nb,Tab )) ;
printf( "Min_==_%f \n" ,MinTab (nb,Tab));
}
```

```
float MaxTab(int N,float Tab[]){
    int i;
    float resultat=Tab[0];
    for(i=1;i<N;i++){
        if(Tab[i]>resultat)
            resultat=Tab[i];
    }
    return resultat;
}
```

```
float MinTab(int N,float Tab[]){
    int i;
    float resultat=Tab[0];
    for(i=1;i<N;i++){
        if(Tab[i]<resultat)
            resultat=Tab[i];
    }
    return resultat;
}
```


Fonctions: Exercice

```
void main()
{
float Tab [100] ;
int nb ;
printf( "Combien_ de_ v a l e u r s _?_");
scanf("%d",&nb) ;
ChargerTab(nb, Tab) ;
EditerTab ( nb,Tab) ;
printf( "Somme_==_%f \n" ,SommeTab(nb,Tab));
printf ( "Moyenne_=_%f \n" ,MoyenneTab(nb ,Tab)) ;
printf ( "Max_====_%f \n" ,MaxTab(nb,Tab )) ;
printf( "Min_====_%f \n" ,MinTab (nb,Tab));
}
```

```
Combien_ de_ v a l e u r s _?_ 4
Valeur_1:_12
Valeur_2:_34
Valeur_3:_2
Valeur_4:_4

Valeur_1_=_12.000000
Valeur_2_=_34.000000
Valeur_3_=_2.000000
Valeur_4_=_4.000000

Somme_==_52.000000
Moyenne_=_13.000000
Max_====_34.000000
Min_====_2.000000
```

Fonctions: Exemple?

```
#ifndef _STDIO_H_
#define _STDIO_H_ /** * file stdio.h */
#include <fcntl.h>
#include <types.h>
#include <stdarg.h>

int printf(const char *format, ...);
```

stdio.h

```
/** * @file printf.c */
#include <stdio.h>
int printf(const char *format, ...)
{
    va_list arg;
    int result;
    va_start(arg, format);
    result = vfprintf(stdout, format, arg);
    va_end(arg);
    return result; }
int vprintf(const char *format, va_list ap) {
    return vfprintf(stdout, format, ap); }
```

printf.c

Input: `printf("Color %s, Number %d, Float %4.2f", "red", 123456, 3.14);`

Output: Color red, Number 123456, Float 3.14

Fonctions: Exemple?

```
#include <stdio.h>

int main()
{
    int c = printf("Hello World");
    printf("\n Nombre de caractres: %d \n",c);
    printf("\n Nombre de caractres: %d ",printf("123456"));
    return 0;
}
```

```
Hello World
  Nombre de caractres: 11
123456
  Nombre de caractres: 6
```